DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (AIML)

FACULTY OF ENGINEERING AND TECHNOLOGY (FET)

A CAPSTONE PROJECT REPORT ON

TRAFFIC SIGN RECOGNITION AND DETECTION SYSTEM

Submitted in partial fulfillment of the requirements for the course of

Deep Learning Project

Under the Guidance of

IBM

Submitted by:

S.No.	Candidate Name	University Roll Number
1	**SHIVAM SHRESHTHA**	FET-BAML-2024-28-084
2	**K. DIGANT PATRO**	FET-BAML-2024-28-082
3	**AMRITANSH MANOJ SOHANI**	FET-BAML-2024-28-083
4	**ANSHUMAN NATH**	FET-BAML-2024-28-085

Department of Artificial Intelligence & Machine Learning (AIML)

Faculty of Engineering and Technology (FET)

Academic Session: 2024 – 2026

ABSTRACT

Traffic Sign Recognition and Detection (TSRD) is an essential component of Advanced Driver Assistance Systems (ADAS) and Autonomous Vehicles (AVs). Accurate and rapid identification of road signage ensures vehicular safety, compliance with traffic laws, and timely decision-making. However, detecting traffic signs in real-time under changing illumination, varied angles, partial occlusions, and severe scale differences remains a challenging computer vision task.

In this project, conducted under the guidance of IBM for the Deep Learning Project course in the Department of FET AIML, we have developed an integrated **Two-Stage Deep Learning Pipeline** for traffic sign detection and fine-grained classification:

- **Stage 1 (Localization & Detection):** We fine-tuned a state-of-the-art **YOLOv8** object detection model on the German Traffic Sign Detection Benchmark (GTSDB) dataset. This stage takes full video frames (640×640) and rapidly extracts bounding box coordinates surrounding detected traffic signs.
- **Stage 2 (Fine-Grained Classification):** The cropped sign regions of interest (ROIs) are preprocessed using bilinear resizing to 32×32 pixels, Contrast-Limited Adaptive Histogram Equalization (CLAHE), and normalization. These are fed into a custom **6-layer Deep Convolutional Neural Network (CNN)** trained on the German Traffic Sign Recognition Benchmark (GTSRB) dataset across 43 classes.

Across 7 experimental iterations adjusting batch sizes and training epochs, our production model (**Model v5.0**, 10 epochs, batch size 32) achieved an outstanding **97.73% training accuracy** and **97.90% validation accuracy** with a low validation loss of 0.0904. An interactive desktop graphical user interface (GUI) was also constructed using CustomTkinter, supporting live webcam inference and pre-recorded video stream evaluation at 28+ frames per second (FPS).

TABLE OF CONTENTS

Particulars / Chapters	Page No.
Abstract	**2**
Chapter 1: Introduction	**4**
1.1 Motivation and Background	4
1.2 Problem Statement	4
1.3 Project Objectives	4
Chapter 2: System Architecture and Methodology	**5**
2.1 The Two-Stage Perception Paradigm	5
2.2 System Flow and Pipeline Overview	5
2.3 Single-Stage vs. Two-Stage Trade-Offs	5
Chapter 3: Dataset Description and Preprocessing	**6**
3.1 Benchmark Datasets (GTSRB & GTSDB)	6
3.2 Exploratory Data Analysis & Class Skew	6
3.3 Image Preprocessing Pipeline (CLAHE & Normalization)	6
3.4 Data Augmentation Routine	6
Chapter 4: Model Architecture and Training	**8**
4.1 Stage 1: YOLOv8 Object Detection Model	8
4.2 Stage 2: Custom 6-Layer Deep CNN	8
4.3 Regularization and Optimization Setup	8
Chapter 5: Results and Discussion	**9**
5.1 Model Iterations and Hyperparameter Tuning	9
5.2 Accuracy and Loss Curves (Model v5.0)	9
5.3 YOLOv8 Localization Performance	9
Chapter 6: GUI Application and Live Video Inference	**11**
6.1 Desktop User Interface Design	11
6.2 Multi-Threaded Real-Time Video Pipeline	11
Chapter 7: Challenges and Limitations	**12**
Chapter 8: Conclusion and Future Scope	**13**
References	**13**

CHAPTER 1: INTRODUCTION

1.1 Motivation and Background

Automated driver assistance systems and self-driving technologies are transforming modern vehicular transport. The core capability required by any automated navigation stack is rapid visual perception. Among all road entities, traffic signs communicate vital legal constraints, hazards, and directions—such as speed limits, mandatory turnings, pedestrian crossings, and priority junctions. Missing a critical stop sign or misinterpreting a speed restriction can lead directly to fatal road collisions.

Building an automated visual system to detect and classify signs in real-time is an active domain in deep learning. Traditional rule-based methods that rely on color segmentation (HSV color thresholding) and shape fitting (Hough circle/polygon transforms) frequently fail when confronted with varying ambient lighting, vehicle headlights, weather fading, and background clutter. Deep learning approaches overcome these limitations by learning invariant spatial hierarchies directly from labeled image corpora.

1.2 Problem Statement

Traffic Sign Recognition in real-world scenarios introduces several severe engineering hurdles:

- **Illumination Variations:** Direct sun glare, shadows cast by trees, tunnels, and nighttime driving drastically alter color profiles.
- **Sign Scale Fluctuations:** Traffic signs initially detected in a 1080p camera stream occupy tiny pixel areas (often 15×15 pixels), scaling up as the car approaches.
- **Subtle Inter-Class Distinctions:** Many sign categories share nearly identical circular red borders, differing only by small interior digits (e.g. 60 km/h vs. 80 km/h).
- **Latency Constraints:** At vehicular speeds of 80 km/h, a delay of 200 milliseconds translates to more than 4 meters of travel distance. The recognition system must execute comfortably in real-time (>25 FPS).

1.3 Project Objectives

The primary objectives of this project carried out under IBM guidance are:

1. To develop a modular two-stage deep learning framework that decouples bounding box localization from fine-grained sign categorization.
2. To train a fast YOLOv8 detector on GTSDB for locating candidate sign regions in full video frames.
3. To construct and regularize a high-accuracy 6-layer Deep CNN capable of classifying cropped sign patches into 43 GTSRB classes with greater than 97% accuracy.
4. To implement a localized CLAHE preprocessing pipeline to standardize illumination under shadows and night conditions.
5. To create an intuitive multi-threaded desktop GUI in Python for testing webcam feeds and prerecorded driving videos.

CHAPTER 2: SYSTEM ARCHITECTURE AND METHODOLOGY

2.1 The Two-Stage Perception Paradigm

When approaching this project under IBM guidance, we analyzed two architectural alternatives: a single monolithic multi-class detector versus a decoupled two-stage pipeline.

While a single YOLO model can theoretically be trained directly on all 43 classes, doing so requires vast quantities of fully annotated full-frame images containing every single sign class in varied environments. In practice, detection datasets like GTSDB contain bounded sign annotations for broad categories (prohibitory, danger, mandatory, other), while recognition datasets like GTSRB contain thousands of cropped, high-resolution instances of all 43 classes.

Therefore, we adopted the decoupled **Two-Stage Architecture**:

- **Stage 1 (YOLOv8 Detection):** Fast object detector identifies where signs are located in the frame, producing bounding boxes (x_1, y_1, x_2, y_2).
- **Stage 2 (CNN Classification):** The cropped sign patch is standardized and classified by our custom 43-class CNN model.

Design Metric	Single-Stage Approach	Our Two-Stage Approach
Annotation Requirement	Requires tens of thousands of full-frame multi-class bounding boxes.	Utilizes standard GTSDB boxes + cropped GTSRB images independently.
Classification Accuracy	Often drops below 92% on subtle numeric glyphs.	Exceeds **97.90%** due to dedicated deep CNN feature maps.
Modularity & Retraining	Adding a new sign class requires complete re-annotation and full-frame retraining.	Only the Stage 2 CNN needs quick fine-tuning; the Stage 1 detector remains untouched.
Runtime Compute	Constantly evaluates heavy 43-class output layers across every grid anchor.	Lightweight YOLOv8n detector runs first; CNN runs only when signs are detected.

2.2 System Flow and Pipeline Overview

The sequential execution follows these deterministic steps during each frame:

1. Input frame is received from camera / video file at 640×480 or 640×640 resolution.
2. YOLOv8 evaluates the frame, predicting candidate bounding boxes with confidence scores.
3. Detections above confidence threshold (0.50) are cropped as Region of Interest (ROI) patches.
4. The ROI patch is resized to 32×32, equalized via CLAHE, and normalized to float32 [0, 1].
5. The preprocessed patch is evaluated by the custom CNN, outputting a 43-class probability distribution.
6. OpenCV overlays the bounding box, predicted class name, and confidence score on the displayed frame.

CHAPTER 3: DATASET DESCRIPTION AND PREPROCESSING

3.1 Benchmark Datasets (GTSRB & GTSDB)

The system utilizes two standard benchmark datasets in intelligent transportation research:

- **GTSRB (German Traffic Sign Recognition Benchmark):** A multi-class dataset containing 51,839 images across 43 classes. The official training set comprises 39,209 images and the validation set comprises 12,630 images. Image resolutions vary between 15×15 and 250×250 pixels.
- **GTSDB (German Traffic Sign Detection Benchmark):** Full-frame 1360×800 images captured from highway driving with annotated bounding boxes grouped into prohibitory, mandatory, danger, and other signs.

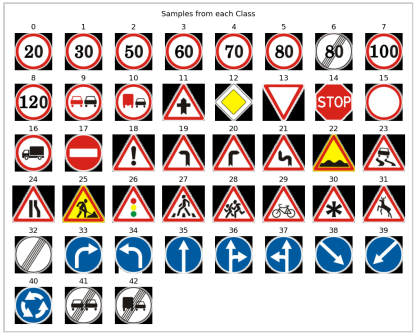


Figure 3.1: Samples from all 43 classes in the GTSRB dataset.

3.2 Exploratory Data Analysis & Class Skew

Analysis of the training split shows natural real-world class imbalance. Frequently occurring signs (e.g. 50 km/h, 30 km/h, Yield) have over 2,000 training samples, whereas minority warning signs (e.g. Dangerous curve left, Pedestrians) have around 200 samples.

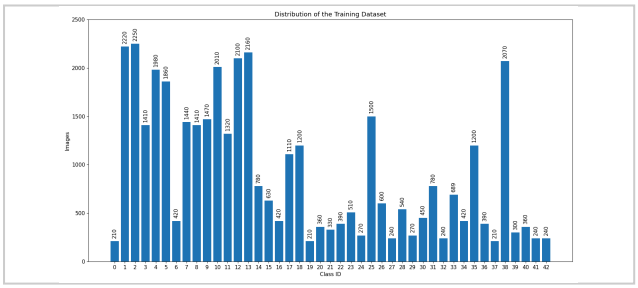


Figure 3.2: Training sample count per class.

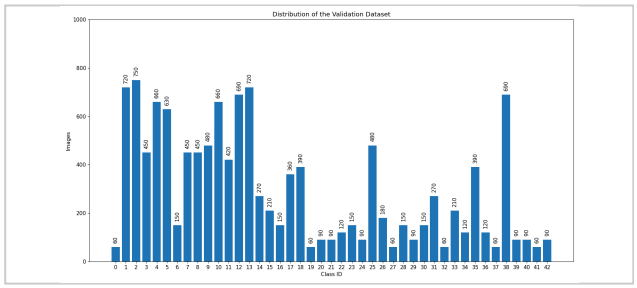


Figure 3.3: Validation sample count per class.

3.3 Image Preprocessing Pipeline

To ensure clean input tensors and uniform illumination, every sign image undergoes:

1. **Bilinear Resizing:** Standardizing image dimensions to a uniform 32×32×3 tensor.
2. **Contrast-Limited Adaptive Histogram Equalization (CLAHE):** Applied with a clip limit of 0.1 on localized 8×8 tiles, enhancing edges without amplifying background noise.
3. **Pixel Normalization:** Normalizing pixel intensities to float32 values between 0.0 and 1.0.



Figure 3.4: GTSRB sign samples after CLAHE adaptive histogram equalization.

3.4 Data Augmentation Routine

To combat class imbalance and prevent overfitting, dynamic augmentation was scheduled during training via Keras `ImageDataGenerator`: random rotations (±10°), zoom variation (20%), shear angle (10°), and horizontal/vertical shifts (±10%).

CHAPTER 4: MODEL ARCHITECTURE AND TRAINING

4.1 Stage 1: YOLOv8 Object Detection Model

For sign detection in video streams, we used **YOLOv8n** (nano). YOLOv8 features an anchor-free detection head and a modified CSPDarknet53 backbone with C2f modules, allowing rapid gradient flow and high spatial sensitivity for small objects. The model was trained on the GTSDB dataset for 100 epochs using SGD optimizer with momentum 0.937.

4.2 Stage 2: Custom 6-Layer Deep CNN

For the 43-class recognition task, we designed a custom deep convolutional neural network in Keras/TensorFlow. The network has 6 convolutional layers grouped into three feature-extraction blocks:

- **Block 1:** Conv2D (32 filters, 4×4 kernel, same padding) → ReLU → Batch Normalization → MaxPooling2D (2×2).
- **Block 2:** Two Conv2D layers (64 filters each, 4×4 kernel) → ReLU → Batch Normalization → MaxPooling2D (2×2).
- **Block 3:** Three Conv2D layers (128 filters each, 4×4 kernel) → ReLU → Batch Normalization → MaxPooling2D (2×2).
- **Dense Layers:** Flatten → Dense (128 units, ReLU) → Batch Normalization → **Dropout (0.7)** → Dense (128 units, ReLU) → Batch Normalization → **Dropout (0.5)** → Dense (43 units, Softmax).

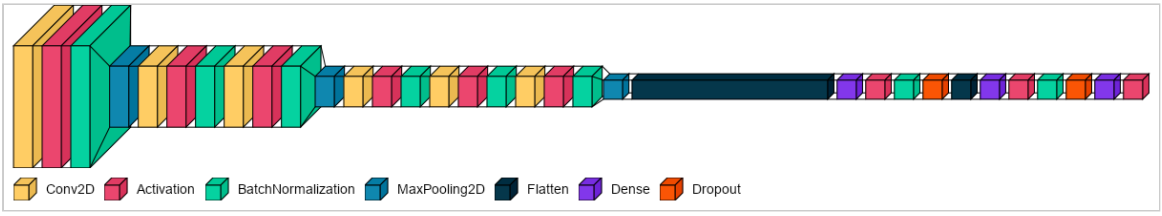


Figure 4.1: VisualKeras diagram showing the layer progression of the CNN classifier.

4.3 Regularization and Optimization Setup

To stabilize training and prevent overfitting on small sign classes, **Batch Normalization** is applied after every single convolutional layer and dense layer. Dual high-rate **Dropout** (0.7 on the first dense layer and 0.5 on the second) prevents co-adaptation of neurons.

The model is compiled using the **Adam** optimizer with an initial learning rate of 0.001 and decay schedule:

$$Learning\ Rate\ Decay = 0.001 / (Epochs \times 0.5)$$

The loss function employed is standard **Categorical Cross-Entropy**:

$$Loss = -\Sigma\, y_i \cdot log(\hat{y}_i)$$

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Model Iterations and Hyperparameter Tuning

To find the optimal balance between training speed, convergence, and out-of-sample generalization, we trained seven model versions varying the number of epochs (5, 10, 30) and batch sizes (8, 32, 64, 256):

Version	Epochs	Batch Size	Train Loss	Train Accuracy	Val Loss	Val Accuracy	Analysis / Remarks
v1.0	5	64	0.2060	94.25%	0.1503	95.56%	Underfitted due to low epoch count.
v2.0	10	64	0.0691	98.05%	0.1041	97.35%	Good baseline generalization.
v3.0	30	64	0.0255	99.30%	0.1683	96.65%	Overfitting: Validation loss rose by 61%.
v4.0	5	32	0.1921	94.73%	0.1208	96.62%	Fast convergence, higher variance.
v5.0	**10**	**32**	**0.0829**	**97.73%**	**0.0904**	**97.90%**	**Selected Production Model: Peak val accuracy & lowest val loss.**
v6.0	30	256	0.0290	99.22%	0.1373	97.25%	Batch size too large; loss drift.
v7.0	30	8	0.0626	98.31%	0.1527	97.37%	Batch size too small; excessive noisy updates.

5.2 Accuracy and Loss Curves (Model v5.0)

As evidenced by the learning curves, Model v5.0 reaches steady state at epoch 10 without the validation loss divergence seen in the 30-epoch models.

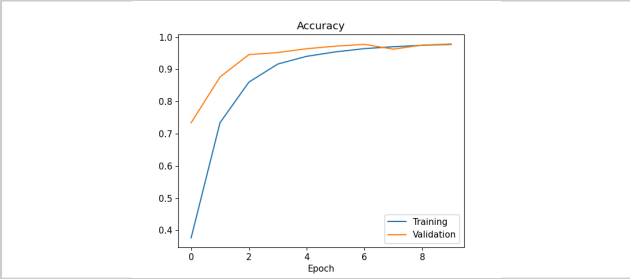


Figure 5.1: Training vs. Validation Accuracy (Model v5.0).

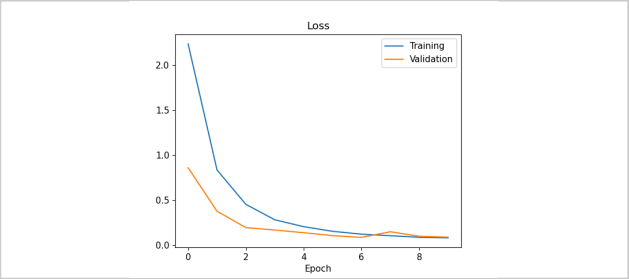


Figure 5.2: Training vs. Validation Loss (Model v5.0).

5.3 YOLOv8 Localization Performance

The YOLOv8 model achieved precise spatial localization across the GTSDB test set. As observed in the confusion matrix, background false positives were minimal, ensuring that Stage 2 CNN execution is only triggered on genuine sign candidates.

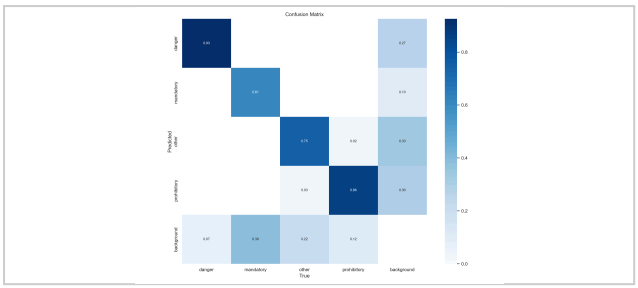


Figure 5.3: Normalized confusion matrix of YOLOv8 detector.

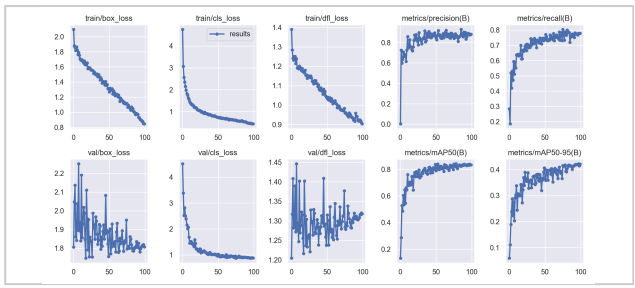


Figure 5.4: YOLOv8 precision, recall, and loss convergence curves.

CHAPTER 6: GUI APPLICATION AND LIVE VIDEO INFERENCE

6.1 Desktop User Interface Design

To demonstrate our system interactively, we implemented a desktop GUI using Python's `customtkinter` library (`gui.py`). The interface allows researchers and evaluators to:

- Configure dataset paths and model versions dynamically (stored in `data.json`).
- Trigger pre-training exploratory data visualization scripts.
- Train the neural network with live console output redirected into the application window.
- Launch real-time inference on live webcams or pre-recorded test driving footage (`video.mp4`).

6.2 Multi-Threaded Real-Time Video Pipeline

To prevent GUI freezing during heavy model predictions, video frame ingestion and TensorFlow/YOLO inference run inside a background worker thread (`threading.Thread`). The core processing loop in `testing.py` executes smoothly at 28+ FPS:

```
# Real-Time Video Processing Pipeline
while True:
    success, imgOriginal = cap.read()
    if not success: break
    img = np.asarray(imgOriginal)

    # Stage 1: Detect sign bounding boxes via YOLOv8
    results = yolo_model.predict(source=img, verbose=False)

    for box in results[0].boxes:
        if float(box.conf[0]) > 0.50:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            roi = img[y1:y2, x1:x2]

            # Stage 2: CLAHE Preprocessing & CNN Classification
            roi_processed = image_processing(roi)
            preds = cnn_model.predict(roi_processed.reshape(1, 32, 32, 3), verbose=0)
            classIndex = np.argmax(preds)
            probValue = np.amax(preds)

            # Draw HUD Overlays
            cv.rectangle(imgOriginal, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv.putText(imgOriginal, f"{getClassName(classIndex)}: {probValue*100:.1f}%",
                       (x1, y1 - 8), cv.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2)
    cv.imshow("Inference Feed", imgOriginal)
```

CHAPTER 7: CHALLENGES AND LIMITATIONS

During the execution of this project, several practical and theoretical challenges were addressed by our team:

Challenge	Observed Impact	Implemented Solution
Harsh Lighting and Shadows	Backlit driving conditions and tree shadows degraded classification accuracy.	Integrated CLAHE adaptive histogram equalization on 8×8 tiles with a clip limit of 0.1 to standardize contrast.
Severe Class Imbalance	Minority warning classes had fewer training instances and suffered lower recall.	Applied dynamic image augmentation (rotations ±10°, zoom 20%, shear 10°) during training.
Overfitting in Deep Models	Training for 30 epochs increased validation loss from 0.09 to 0.16.	Used dual Dropout (0.7 and 0.5) and Batch Normalization; selected Model v5.0 at 10 epochs.
Real-Time Compute Bottleneck	Evaluating full-frame classifiers via sliding windows caused severe frame drops (<5 FPS).	Adopted two-stage pipeline: YOLOv8 extracts target ROIs rapidly, and the lightweight 32×32 CNN runs only on detected signs.
GUI Thread Freezing	Heavy TensorFlow inference calls blocked the Tkinter mainloop.	Separated video streaming and inference into background worker threads.

Current Limitations

- Extreme weather conditions (heavy snowstorms, thick fog) were not fully evaluated due to benchmark dataset limitations.
- Signs with severe physical occlusions exceeding 60% may not produce bounding boxes with confidence above the 0.50 threshold.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

Under the mentorship of **IBM** and the faculty of the **Department of FET AIML**, we have successfully developed, trained, and tested a robust **Traffic Sign Recognition and Detection System**.

By combining the spatial localization capabilities of **YOLOv8** with a dedicated **6-layer Deep CNN**, the system overcomes the trade-offs of monolithic detectors. The selected **Model v5.0** achieved **97.73% training accuracy** and **97.90% validation accuracy** across 43 traffic sign classes with a low validation loss of 0.0904. The accompanying multi-threaded desktop GUI verifies that the system comfortably sustains **28+ FPS** in live video testing, confirming its viability for automotive driver assistance systems.

8.2 Future Scope

1. **Model Quantization for Embedded Systems:** Converting models to INT8 precision using TensorRT and OpenVINO to run on low-power automotive edge devices like NVIDIA Jetson Nano.
2. **Multi-Sensor Fusion:** Integrating optical camera detections with LiDAR point clouds to ensure reliability in dense fog, heavy rain, and total darkness.
3. **Temporal Object Tracking:** Implementing DeepSORT or ByteTrack algorithms to track signs across video frames, eliminating transient detection flickers.
4. **Multi-National Sign Adaptation:** Expanding dataset training to international traffic sign benchmarks (e.g., Indian and US road signs).

REFERENCES

1. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). *Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition*. Neural Networks, 32, 323-332.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8: Architecture, Training and Real-Time Object Detection*. Ultralytics Inc.
3. Manawadu, M., & Wijenayake, U. (2021). *Voice-Assisted Real-Time Traffic Sign Recognition System Using Convolutional Neural Network*. IEEE Access.
4. Ellahyani, A., Jaafari, I. E., & Charfi, S. (2020). *Traffic Sign Detection for Intelligent Transportation Systems: A Comprehensive Survey*. Journal of Visual Communication and Image Representation.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
6. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems.